# System Reset: A Forensic Audit and Strategic Architecture Overhaul of the Stila Framework

## Executive Summary: The Architecture of Bureaucratic Collapse

The Stila framework, initially conceived as a "repo-resident AI operating system," has suffered a catastrophic architectural failure. This failure is not one of capability or intent, but of complexity and structural over-determinism. The repository snapshot reveals a system that has metastasized from a lightweight collaboration tool into a dense bureaucratic regime, characterized by "Process Theater" and "Cognitive Overload." This report constitutes a forensic audit of the current operational impasse and provides a strategic roadmap for "Process Bankruptcy"—a deliberate, strategic elimination of debt to restore operational liquidity.

The analysis indicates that the project suffers from "Context Collapse"—a phenomenon where the sheer volume of governance documentation (ops/, docs/library/, protocols/) overwhelms the agent's context window, degrading its ability to reason about the actual code.[1] Furthermore, the imposition of human-centric organizational metaphors (Contractors, Foremen, Dispatch Packets) upon a probabilistic token-prediction model has induced "Mode Confusion," where the agent cannot distinguish between conversational ideation and rigid execution.[3] The agent is trapped in a double-bind of conflicting system prompts, attempting to be both a high-level strategist and a low-level tactician within the same inference stream.

The current state, documented in STATE_OF_PLAY.md, reflects a frantic iteration cycle (DP-OPS-0001 through DP-OPS-0050) attempting to patch behavioral drift with increasing layers of explicit rules. This is a classic "spiral of over-formalization," where the solution to a failed rule is interpreted as the need for *more* rules, rather than *better* constraints.[5] The "Anti-Drift" doctrine has paradoxically become the primary source of drift, as the maintenance of the governance layer now consumes more energy than the execution of the actual work.

The recommendation is not optimization, but demolition. We must dismantle the Panopticon of ops/ and revert to a Minimum Viable System (MVS) that leverages the raw competency of modern Large Language Models (LLMs) grounded by specific file context, rather than performative protocols.

# Part I: The Forensic Audit of System Failure

## 1.1 The Taxonomy of Bloat: Analyzing the Repository Structure

A review of the repository structure provided in the snapshot [7] reveals the physical manifestation of the system's failure. The ops/ directory, intended to be the "kernel" of the AI operating system, has become a labyrinth of conflicting directives and redundant abstractions. This density is the primary driver of both the agent's context confusion and the operator's cognitive load.

### The "Ops" Directory: A Case Study in Over-Engineering

The ops/ directory contains 56 distinct files.[7] In a repository where public_html represents the actual value, the existence of 56 governance files represents a staggering ratio of overhead to payload. This directory has become a "bureaucracy simulation" rather than an operating system.

#### 1. Redundant Constitutions and Conflicting Truths

The system maintains multiple "Source of Truth" documents, creating a fragmented constitutional landscape.

- **PROJECT_TRUTH.md**: Declares itself the primary constitution ("If something conflicts with this file, this file wins").[7]
- **AGENTS.md**: Listed as a "pointer-first agent constitution".[7]
- **ops/contracts/**: Contains further binding agreements like CONTRACTOR_DISPATCH_CONTRACT.md and OUTPUT_FORMAT_CONTRACT.md.[7]

This requires the agent to reconcile three distinct constitutional layers before processing a single user request. LLMs struggle with "Instruction Hierarchy" when conflicting or redundant instructions are distributed across multiple files in the context window. The agent must expend attention mechanism resources to determine which "Truth" is currently applicable, leading to "Context Distraction".[8] The result is not higher fidelity, but lower adherence, as the model "averages out" the conflicting constraints.

**2. Performative Protocols (Process Theater)** The directory includes files such as ops/init/protocols/SAVE_THIS_PROTOCOL.md and ops/init/protocols/UI_MODE_PROTOCOL.md.[7] These represent "Process Theater"—the codification of behaviors that should be implicit interactions into formal rituals.[9]

- *The Failure:* Defining a protocol for "Saving" implies that the agent lacks the basic capability to save data without a governance framework. This is false. Modern LLMs have innate capabilities for information retention within the context window. Explicitly documenting these "breathing instructions" consumes valuable token space (context window) with low-entropy information, pushing high-value code context out of the

effective attention span.[10]

- *The Cost:* Every time the operator has to reference or enforce the UI_MODE_PROTOCOL, they are engaging in meta-work. This aligns with findings that over-formalization in agile environments leads to "process fatigue" without delivering actual agility.[11]

**3. The "Mementos" and "Stelae" Fallacy** The inclusion of docs/library/MEMENTOS.md (bias artifacts) and docs/library/STELAE.md (operator nudges) [7] introduces a "soft" governance layer that competes with the "hard" constraints of PROJECT_TRUTH.md.

- *The Mechanism:* This nuance—distinguishing between a "hard constraint" (PROJECT_TRUTH) and a "soft preference" (MEMENTOS)—is computationally expensive for an LLM. The attention mechanism effectively flattens all input tokens. The agent does not "feel" the sociological weight of a Stela versus a Contract; it merely processes conflicting probabilities.
- *The Result:* The agent creates a "blended" behavior where it might prioritize a "soft" stylistic preference from Mementos over a "hard" functional constraint from Project Truth, simply because the Memento was phrased more uniquely or appeared later in the context stream. This contributes to the "instability" reported by the user.[5]

### The "Docs" Directory: Bureaucracy as Content

The docs/ directory mirrors the complexity of ops/, further fragmenting the knowledge base.

- **Fragmented Onboarding:** The presence of docs/10-QUICKSTART.md, docs/START_HERE.md, docs/README_CONTEXT.md, and docs/CONTRACTOR_PACKET.md [7] creates a fractured entry point. An agent rehydrating context must parse four different files to understand how to begin. Research on "Context Engineering" emphasizes the need for a "Single Source of Truth" to prevent "context clash".[12] By distributing onboarding across four files, Stila ensures that the agent constructs a fragmented mental model of the project's entry procedures.
- **Archival Hoarding:** The directory structure includes docs/_archive/ and docs/triage/INBOX.md, encouraging the retention of obsolete state. This violates the "Minimum Viable Data" principle.[13] Archival data in the active context window acts as "distractor tokens," reducing the model's retrieval accuracy for relevant, current information.[14]

## 1.2 The Mechanism of Mode Failure

The user reports that the AI "frequently blends discussion and execution" and "produces artifacts when discussion was requested." This is identified as **Mode Confusion** [3], a failure mode endemic to single-stream agent architectures that attempt to compress distinct cognitive phases into a linear chat history.

### The "Single-Thread" Fallacy

Stila attempts to force a single chat thread to serve as Planner, Executor, and Verifier simultaneously.

- **Conflicting System Prompts:** The "Architect" persona (strategy) and the "Foreman" persona (tactics) are active in the same context.[7] The agent is instructed to "DO NOT write low-level implementation code" (Architect) but also to "Handle the actual file writing" (Foreman). While the prompt attempts to delegate these roles to different *virtual* entities (Gemini vs. ChatGPT), the *context window* is shared. The model sees a history of high-level strategy mixed with low-level diffs.
- **Token Probability Contamination:** LLMs function as next-token predictors. If the context history contains a mix of high-level strategy (audit/plan) and low-level code (execution), the probability distribution for the next token becomes "muddy".[3] The model will oscillate between chatting about the plan and trying to write the code because the historical pattern contains both. This explains why the agent "produces artifacts when discussion was requested"—the presence of artifacts in the history increases the probability of generating a new artifact, regardless of the immediate instruction.[15]

### The "Discuss-Only" Patch

The introduction of DISCUSS-ONLY cues [7] is a symptom of this failure. It is a "patch" attempting to software-lock a behavior that should be architecturally isolated.

- *The Failure:* If an agent requires a verbal cue to stop it from writing code, the architecture has failed to separate the *Planning* state from the *Execution* state. Research into "Planner-Executor" frameworks demonstrates that these modes function best when completely isolated—the Planner should not even have access to the code-writing tools, and the Executor should not be tasked with high-level reasoning.[16] By mixing them, Stila forces the model to rely on "self-restraint" (a weak capability) rather than architectural constraints.

### Context Collapse and "Lost in the Middle"

The ops/init/icl/ directory contains massive context packs (ICL_CONTINUITY_CORE.md, DAILY_CONSOLE.md, CONTEXT_PACK.md). As the session progresses, this initial instruction block is pushed further back in the context window.

- **Instruction Decay:** LLMs exhibit "attention decay" over long contexts, often prioritizing the beginning (system prompt) and the immediate end (recent messages), while losing information in the middle.[18] The rigorous rules defined in PROJECT_TRUTH.md at the start of the session become less weighted than the immediate conversational nuances at the end. The agent "drifts" because the "Anti-Drift" doctrine is mathematically less salient than the user's most recent query.
- **Recursive Summarization Failure:** The reliance on STATE_OF_PLAY.md as a ledger attempts to mitigate this, but as the ledger grows, it too contributes to context overload.[19] The system is fighting fire with flammable materials—attempting to cure context loss by

adding more text.

## 1.3 Operator Burnout: The Cognitive Load Crisis

The operator reports feeling like they are "Operating the AI instead of collaborating with it." This is a textbook case of **Extraneous Cognitive Load** overshadowing Germane Cognitive Load.[20]

### The Cost of Meta-Work

The operator is forced to manage the *process* of the AI rather than the *product*.

- **The Protocol Compliance Tax:** Tasks such as checking DP-OPS numbers, verifying RECEIPT formats, enforcing STELAE compliance, and policing STATE_OF_PLAY updates create a massive "Cognitive Overhead".[22] The user is spending more mental energy verifying that the AI followed the *rules of the game* than verifying if the code actually works.
- **Semantic Fragility:** The system requires "exact phrasing" (e.g., the "Approval Phrase" APPROVE <DP-ID> EMIT DB-PR-META [7]). This fragility forces the operator to memorize rigid incantations. If they miss a keyword, the system stalls. This turns the natural language interface—the primary benefit of LLMs—into a rigid Command Line Interface (CLI) with worse error reporting. This is "Process Theater" at its worst: a simulation of rigor that adds no actual safety.[9]

# Part II: The Brutal Simplification Proposal

The current system is insolvent. The following section outlines the specific deletion, merger, and demotion of repository assets to resolve the Impasse. This is not a reorganization; it is a liquidation.

## 2.1 The Kill List (Delete Immediately)

The following files and directories create friction without adding verifiable value. They must be deleted to reduce context noise and operator burden.

| File / Directory | Action | Rationale |
|---|---|---|
| **AGENTS.md** [7] | **DELETE** | Redundant with PROJECT_TRUTH. Maintaining two constitutions creates alignment drift. The "Pointer-first" concept is |

| | | |
|---|---|---|
| | | unnecessary overhead. |
| **docs/library/STELAE.md** [7] | **DELETE** | "Operator nudges" are meta-governance. If a rule is critical, it belongs in the prompt. If it is a "nudge," it is noise. The operator does not need a library of "reminders to stay calm." |
| **docs/library/MEMENTOS.md** [7] | **DELETE** | "Bias artifacts" and "soft preferences" confuse the model. The model needs binary constraints (Do/Don't), not stylistic "vibes." |
| **ops/init/icl/AI_CONTEXT_SYNC.md** [7] | **DELETE** | Manual context sync is toil. The repo itself is the sync point. This file represents "Process Theater"—pretending to sync context that should be derived from git log. |
| **ops/init/icl/DAILY_CONSOLE.md** [7] | **DELETE** | Diaries are not ops. "Today's intent" belongs in the specific prompt, not a committed file. This encourages "Log vs. Canon" confusion. |
| **ops/contracts/CONTRACTOR_DISPATCH_CONTRACT.md** [7] | **DELETE** | The rules "No direct pushes" and "PR-only" are already in CONTRIBUTING.md. This file is redundant legalism. |
| **ops/init/protocols/UI_MODE_PROTOCOL.md** [7] | **DELETE** | The AI should adapt to the provided text interface. |

| | | |
|---|---|---|
| | | Codifying "UI Modes" adds unnecessary conceptual entities. |
| **ops/init/protocols/SAVE_THIS_PROTOCOL.md** [7] | **DELETE** | Saving relevant info is a basic competency, not a protocol. |
| **docs/triage/INBOX.md** [7] | **DELETE** | "Capture ideas fast" is what issue trackers (GitHub Issues) are for. Committing messy thoughts to the repo pollutes the "Source of Truth." |
| **ops/init/manifests/ (All files)** | **DELETE** | The file system *is* the manifest. Listing files in a text file (CONTEXT_MANIFEST.md) creates a maintenance burden where the map drifts from the territory. |

## 2.2 The Consolidation (Merge and Purge)

The remaining "Truth" must be centralized into a single, indivisible atomic unit to minimize context switching and instruction hierarchy issues.

1. **Merge PROJECT_TRUTH.md, PROJECT_MAP.md, and CANONICAL_TREE.md** into **CONTEXT.md**.
    - *Reasoning:* The agent currently has to "read-in" three different files to understand what the project *is* and where things *are*. A single CONTEXT.md provides a unified world-view. This aligns with the "Single Source of Truth" pattern advocated in high-performance DevOps.[23]
2. **Merge ops/init/protocols/DISPATCH_PACKET_PROTOCOL.md** and **ops/templates/DISPATCH_PACKET_TEMPLATE.md** into **templates/TASK.md**.
    - *Reasoning:* "Dispatch Packet" is over-formalized jargon. "Task" is universal. The protocol (rules) should be embedded as comments within the template (structure), ensuring the rules are present exactly when the structure is used. This enforces "Just-in-Time" governance rather than "Just-in-Case" bureaucracy.
3. **Merge docs/10-QUICKSTART.md, docs/20-GOVERNANCE.md, CONTRIBUTING.md** into **README.md**.

- *Reasoning:* The README.md should be the *only* entry point. Segmenting governance into sub-files ensures the agent (and operator) misses critical rules if they don't traverse the full tree.

### 2.3 The Demotion (Canon to Reference)

Files that are retained but stripped of "Constitutional" weight.

- **STATE_OF_PLAY.md: DEMOTE.**
  - *New Role:* This is no longer a "Dashboard" that requires "Policing".[7] It is a simple changelog. The "State of Play Policing" workflow [7] should be deleted. The Git log is the source of truth for history; STATE_OF_PLAY.md is merely a human-readable summary.
- **docs/library/: DEMOTE/ARCHIVE.**
  - *New Role:* Move useful technical documentation (like GEOIP_IMPORTER.md) to a wiki/ or reference/ folder. Stop treating them as "curated operator surfaces."

### 2.4 The Preservation (Keep at All Costs)

1. **ops/bin/snapshot (and repo2txt functionality): KEEP.**
   - *Reasoning:* The ability to dump the repository state into a text stream is the *only* technical advantage of this system. It is the lifeline for the AI's context.[24]
2. **public_html/ and src/: KEEP.**
   - *Reasoning:* The runtime code.
3. **.github/workflows/repo_gates.yml: KEEP.**
   - *Reasoning:* Automated verification (CI) is the only "hard" constraint that matters. Everything else is just text. If the code passes the tests, the text description matters less.

## Part III: The Minimum Viable System (MVS)

**"What is the smallest version of Stila that actually works and reduces human burden?"**

The MVS consists of **One Context File**, **One Task Template**, and **Two Scripts**.

### 3.1 The Artifacts

1. **CONTEXT.md (The Brain):**
   - Contains: Project goal, Directory map (simplified), Coding Standards (formerly PROJECT_TRUTH), and current high-level status.
   - Size limit: < 2,000 tokens. This forces discipline in what is considered "truth."
2. **TASK.md (The Handshake):**
   - A simple markdown template for defining a unit of work.
   - Sections: Goal, Context Files (paths), Plan (steps), Verification (test command).
3. **bin/dump (The Eyes):**

    - A simplified version of ops/bin/snapshot. Outputs the content of files listed in TASK.md + CONTEXT.md.
4. **bin/apply (The Hands):**
    - A script to apply a standard git patch or simply guide the git apply process.

### 3.2 The Workflow (The Loop)

This workflow replaces the 10-step "Dispatch Packet" conveyor belt with a 6-step standard engineering loop.

1. **Draft:** Operator writes a TASK.md describing the bug or feature.
2. **Dump:** Operator runs bin/dump TASK.md > prompt.txt.
3. **Run:** Operator pastes prompt.txt into the AI model (The Planner or Executor).
4. **Result:** AI returns a code block (diff or file content) and a verification command.
5. **Verify:** Operator applies code, runs verification.
6. **Commit:** Operator commits.

**Why this works:** It removes "Dispatch Protocols," "Receipts," "Approval Phrases," and "Stelae." It relies on the raw competency of the model grounded by specific file context. It shifts the burden from *formatting* to *verifying*.

## Part IV: Diagnosing and Fixing Mode Failure

The prompt asks: *Why does the AI keep blending discussion and execution? Is this solvable with structure alone? Or does it require role separation?*

### 4.1 The Diagnosis: Single-Stream Cognitive Overload

The AI is blending discussion and execution because **Stila asks it to be two things at once**.

- In the current DISPATCH_PACKET_PROTOCOL, the agent is asked to "Audit" (Planner) and "Execute" (Worker) in the same conversational stream.
- LLMs function as "Next Token Predictors." If the context contains a mix of high-level strategy (audit/plan) and low-level code (execution), the probability distribution for the next token becomes "muddy." It will oscillate between chatting about the plan and trying to write the code.[3] The model's "latent state" cannot hold the distinction between "thinking about code" and "writing code" when the history is a jumble of both.

### 4.2 The Solution: Architectural Role Separation (Planner-Executor Pattern)

Structure alone (better prompts) is **not** sufficient. The failure is architectural. You must implement a **Planner-Executor** pattern.[16] This pattern is proven to reduce error rates and improve adherence to constraints by isolating the cognitive modes.

- **Session A (The Planner):**

    - **Input:** CONTEXT.md + User Request.
    - **Constraint:** *Read-Only.* No code generation allowed.
    - **Output:** A TASK.md file (The Plan).
    - *Mechanism:* The AI acts as the "Architect." It explores the codebase (via dump), identifies the files to change, and writes the *specification* for the change. It *cannot* write the code. This enforces "Thinking Time" without the pressure of syntax.
- **Session B (The Executor):**
    - **Input:** TASK.md (The Plan) + Content of files listed in the Plan.
    - **Constraint:** *Write-Only.* No discussion allowed.
    - **Output:** Code / Diffs / Patches.
    - *Mechanism:* The AI acts as the "Contractor." It takes the spec and the specific files and produces the implementation. It is forbidden from re-planning. This reduces "Mode Confusion" to zero because the Planner is not present in the Executor's context.

**Diagnosis:** The "Mode Confusion" is a symptom of **Context Contamination**. By engaging in discussion *and* execution in the same context window, the operator contaminates the "execution state" with "discussion noise."

**Fix:** Different sessions. Different system prompts. The "Handoff" is a file (TASK.md), not a chat message.

# Part V: The Reset Strategy

The operator is experiencing burnout. Incremental changes will feel like "more work." We need a **Hard Reset**.[27]

## 5.1 The "Bankruptcy" Pattern

We will declare "Process Bankruptcy." This is a psychological and technical reset.

1. **Archive:** Create a folder archive/stila_v1/. Move ops/, docs/library/, and AGENTS.md into it. They are dead. Do not delete them yet (safety blanket), but move them out of the active path.
2. **Freeze:** Stop all feature work. The repository is closed for business for 48 hours.
3. **Rebuild:** Create the MVS artifacts (CONTEXT.md, TASK.md) from scratch. Do not copy-paste the old complex rules. Write them fresh, max 1 page each.

## 5.2 The "Tabula Rasa" Protocol

For the AI to respect the reset, we must break its pattern matching.

- **New System Prompt:** Write a radically simple system prompt for the AI. Do not use the words "Stila," "Dispatch Packet," or "Integrator." Use new terms: "Repository Context," "Work Unit," "Developer." This prevents the AI from triggering its training on the old,

complex Stila prompts (if they were part of RAG or long-term memory) or simply falling back into the specific, convoluted "Stila" style it has "learned" from the 50 DPs in the history.
- **Context Clearing:** Force a hard context reset on the AI interface. Delete old chats. Start fresh.

# Part VI: The Strategic Action Plan

## Phase 1: Stabilization (Days 1-3) - "The Purge"

- **Action 1 (Delete):** Execute the "Kill List." Remove the 56 files in ops/ and the "Curated Library."
- **Action 2 (Consolidate):** Create CONTEXT.md in the root. Copy the essential *facts* (not rules) from PROJECT_TRUTH and PROJECT_MAP.
- **Action 3 (Simplify):** Replace docs/10-QUICKSTART.md with a 10-line "How to Work" section in README.md.
- **Action 4 (Tooling):** Simplify ops/bin/snapshot to bin/context. Make it output *only* CONTEXT.md and a user-specified file list. No scopes, no formatting flags.

## Phase 2: Redesign (Days 4-10) - "The Split"

- **Action 1 (Define Planner):** Create a prompt for the "Planner" persona. "You are a Software Architect. You analyze requirements and codebase context. You output a TASK.md file listing specific file paths and changes required. You do not write code."
- **Action 2 (Define Executor):** Create a prompt for the "Executor" persona. "You are a Senior Developer. You receive a TASK.md and file contents. You output git diff blocks. You do not discuss."
- **Action 3 (Pilot):** Run *one* task using this split flow. Measure "Cognitive Load" (did the operator have to argue with the AI? Did the AI verify its own work?).

## Phase 3: Decision Point (Day 14) - "Continue or Stop"

- **Review:** If the operator still feels like they are "Operating the AI," the project concept (Repo-Resident AI) is flawed for this team/codebase combination.
- **Pivot:** If the MVS works, proceed. If not, revert to standard "Copilot" usage (IDE-based autocomplete) and abandon the "Agentic Framework" entirely. The "sunk cost" of Stila v1 must be ignored.

# Detailed Analysis of "Repo-Resident" Failure Modes

## The "Drift" Paradox

The user prompt highlights "Drift" as a major enemy. Stila attempted to fight drift with **Static Canon** (PROJECT_TRUTH.md).

- *Insight:* Static text cannot fight dynamic entropy. The more you write in PROJECT_TRUTH, the *more* drift you create, because the code evolves faster than the documentation.[29]
- *Correction:* Drift is fought with **Tests**, not Text. The repo-gates (CI/CD) are the only effective Anti-Drift mechanism. The MVS prioritizes *executable verification* (tests) over *declarative verification* (truth docs).

### The "Cognitive Overhead" Equation

The current equation is: Effort = (Task Difficulty) + (Stila Protocol Compliance).

Since Stila Protocol Compliance is non-zero and high (checking DPs, formatting Receipts, updating State of Play), the AI is net-negative for simple tasks.

- *Correction:* The MVS removes the Stila Protocol Compliance term. The AI output should look like a standard Pull Request description, not a "Dispatch Packet Receipt." We align with industry standards (Git, Markdown, PRs) rather than inventing a proprietary Stila-Language.

## Conclusion

The Stila framework failed because it optimized for **auditability** at the expense of **usability**. It built a Panopticon for the AI, but the operator ended up trapped in the guard tower, exhausted by the surveillance.

The recommendation is to dismantle the Panopticon. Trust the git log for auditability. Trust CI/CD for safety. Use the AI as a tool (a hammer), not a colleague (a Contractor). The "Agent" metaphor has been taken too literally, resulting in a bureaucracy of simulated employees. Reverting to a "Tool" metaphor—where the AI is a sophisticated text transformation engine driven by precise inputs—will restore control and sanity.

**Immediate Next Step:** Delete ops/. Create CONTEXT.md. Breathe.

---

### Appendix A: The "Brutal Simplification" File Manifest

| Path | Action | Rationale |
|---|---|---|
| ops/ (Entire Directory) | **DELETE** | Root cause of over-formalization. Contains 50+ files of "Process Theater." |
| docs/library/STELAE.md | **DELETE** | Performative complexity. Nudges belong in the |

| | | |
|---|---|---|
| | | system prompt, not a file. |
| docs/library/MEMENTOS.md | **DELETE** | Confusing distinction between "hard" and "soft" rules. |
| AGENTS.md | **DELETE** | Redundant with PROJECT_TRUTH. |
| PROJECT_TRUTH.md | **MERGE** | Content moves to CONTEXT.md. |
| PROJECT_MAP.md | **MERGE** | Content moves to CONTEXT.md. |
| CANONICAL_TREE.md | **MERGE** | Content moves to CONTEXT.md. |
| STATE_OF_PLAY.md | **ARCHIVE** | Replace with standard CHANGELOG.md or git history. |
| .github/copilot-instructions.md | **SIMPLIFY** | Reduce to: "Read CONTEXT.md. Follow patterns. No direct main push." |
| public_html/ | **KEEP** | Runtime code. |
| upstream/ | **KEEP** | Provenance data. |

### Appendix B: The MVS "Task Ticket" Template

*(Replace the 200-line Dispatch Packet with this)*

# TASK:

## Context

[Link to or Paste relevant file paths here]

# Objective

[1-2 sentences on what needs to happen]

# Plan

# Verification

- [ ] Automated: npm test or php artisan test
- [ ] Manual: [Check X in browser]

## Appendix C: The Planner-Executor Prompt Split

**Prompt 1: The Planner**

> "You are the Architect. Read CONTEXT.md. Analyze the User Request. Do not write code. Output a TASK.md plan that lists the specific files to touch and the verification steps required."

**Prompt 2: The Executor**

> "You are the Builder. Read TASK.md. Read the provided file contents. Output the code changes required to satisfy the plan. Do not deviate. Do not explain high-level strategy. Just build."

This report constitutes the requested "external reset." The system is currently insolvent; immediate liquidation of process debt is the only path to recovery.

**Works cited**


1. Context collapse in AI systems - MITRIX Technology, accessed January 22, 2026, https://mitrix.io/blog/context-collapse-in-ai-systems/
2. Context Collapse in AI Systems - by Andrew Whitehouse - Medium, accessed January 22, 2026, https://medium.com/@andrewswhitehouse/context-collapse-in-ai-systems-7374c31a8bc2
3. Alignment, Agency and Autonomy in Frontier AI: A Systems Engineering Perspective - arXiv, accessed January 22, 2026, https://arxiv.org/html/2503.05748v1
4. Risk Perception in Complex Systems: A Comparative Analysis of Process Control and Autonomous Vehicle Failures - MDPI, accessed January 22, 2026, https://www.mdpi.com/2673-2688/6/8/164
5. 7 Signs Your AI Agent is Failing in Production and What to Do, accessed January

22, 2026, https://www.getmaxim.ai/articles/7-signs-your-ai-agent-is-failing-in-production-and-what-to-do/
6. “Reimagining Institutions in the Age of AI Agents” workshop at - BOOST, accessed January 22, 2026, https://boostimpact.org/wp-content/uploads/2025/09/Workshop-Materials-Reimagining-Institutions-in-the-Age-of-AI-Agents-.pdf
7. snapshot-platform-main-01a83e76.txt
8. How Long Contexts Fail, accessed January 22, 2026, https://www.dbreunig.com/2025/06/22/how-contexts-fail-and-how-to-fix-them.html
9. Process Theater vs. Technical Excellence: A Recurring Software Crisis - The New Stack, accessed January 22, 2026, https://thenewstack.io/process-theater-vs-technical-excellence-a-recurring-software-crisis/
10. OpenCode Token Usage: How It Works and How to Optimize It - TrueFoundry, accessed January 22, 2026, https://www.truefoundry.com/blog/opencode-token-usage-how-it-works-and-how-to-optimize-it
11. Agile Software Development in 2025: Evolution, Trends & Future Outlook - Medium, accessed January 22, 2026, https://medium.com/@devxplore/agile-software-development-in-2025-evolution-trends-future-outlook-f4402f14d1f6
12. Context engineering: Best practices for an emerging discipline | Redis, accessed January 22, 2026, https://redis.io/blog/context-engineering-best-practices-for-an-emerging-discipline/
13. Why Minimum Viable Data Is Essential for Protected Agentic AI - K2view, accessed January 22, 2026, https://www.k2view.com/blog/agentic-ai-data-protection
14. How to Optimize Context Windows So Your AI Agents Handle Complex Information, accessed January 22, 2026, https://datagrid.com/blog/optimize-ai-agent-context-windows-attention
15. The End of Infinite Context: Engineering Reliability in the Age of Agentic Workflows | by Ali moradi | Dec, 2025 | Medium, accessed January 22, 2026, https://medium.com/@moradikor296/the-end-of-infinite-context-engineering-reliability-in-the-age-of-agentic-workflows-9531163159b4
16. Planner-Executor Agentic Framework - Emergent Mind, accessed January 22, 2026, https://www.emergentmind.com/topics/planner-executor-agentic-framework
17. Separating AI Agents into Planner and Executor | by Mahdi Jaouadi | Medium, accessed January 22, 2026, https://medium.com/@jaouadi.mahdi1/separating-ai-agents-into-planner-and-executor-7705b58d79fd
18. [2512.14391] RePo: Language Models with Context Re-Positioning - arXiv,

accessed January 22, 2026, https://arxiv.org/abs/2512.14391
19. Techniques for Summarizing Agent Message History (and Why It Matters for Performance) : r/AI_Agents - Reddit, accessed January 22, 2026, https://www.reddit.com/r/AI_Agents/comments/1n6lo58/techniques_for_summarizing_agent_message_history/
20. Understanding Workers' Well-Being and Cognitive Load in Human-Cobot Collaboration: Systematic Review - NIH, accessed January 22, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12423613/
21. Reducing Cognitive Load: The Missing Key to Faster Development Cycles - Agile Analytics, accessed January 22, 2026, https://www.agileanalytics.cloud/blog/reducing-cognitive-load-the-missing-key-to-faster-development-cycles
22. Cognitive Load in the Age of AI: Rethinking Developer Workflows - ITPro Today, accessed January 22, 2026, https://www.itprotoday.com/software-development/cognitive-load-in-the-age-of-ai-rethinking-developer-workflows
23. Single Source of Truth — 5 SSOT Examples You Need | Perforce Software, accessed January 22, 2026, https://www.perforce.com/blog/vcs/single-source-of-truth-examples-ssot
24. RepoSwarm: Give AI Agents Context Across All Your Repos - Robot Paper, accessed January 22, 2026, https://robotpaper.ai/reposwarm-give-ai-agents-context-across-all-your-repos/
25. I created a script to dump entire Git repos into a single file for LLM prompts - Reddit, accessed January 22, 2026, https://www.reddit.com/r/ChatGPTCoding/comments/1fbsetf/i_created_a_script_to_dump_entire_git_repos_into/
26. How to Build a Plan-and-Execute AI Agent - Ema, accessed January 22, 2026, https://www.ema.co/additional-blogs/addition-blogs/build-plan-execute-agents
27. When AI coding tools slow you down (and how to fix it) | by Alek Dobrohotov | Medium, accessed January 22, 2026, https://medium.com/@aleksandardobrohotov/when-ai-coding-tools-slow-you-down-and-how-to-fix-it-3d1e8124bea7
28. How to Reset LLM Context and Refresh Prompts: Quick Step-by-Step Guide - Skywork.ai, accessed January 22, 2026, https://skywork.ai/blog/how-to-reset-llm-context-refresh-prompts-guide/
29. Cognitive Load: Rethinking Human-AI Synergy in the Age of AI Collaboration - Shep Bryan, accessed January 22, 2026, https://www.shepbryan.com/blog/cognitive-load-ai
30. AI agents can act unpredictably without proper guidance, accessed January 22, 2026, https://dig.watch/updates/ai-agents-can-act-unpredictably-without-proper-guidance